

AI Infrastructure Resource Ledger: The Reality Check

Thursday, 5. March 2026

Engineer(s): _____ Zakariya Laouar _____

Date: _____ April 23 _____

Project Title: _____ Long horizon planning with learned world models _____

Phase 1: The Calendar (Working Backward)

If you are still writing your training loop on your Drop-Dead Date, your project will fail.

1. Final Demo / Presentation Date: _____ April 23 _____
2. Days required for Evaluation & Report: 4 days.
3. Estimated Training Run Duration: 2 days.
4. DROP-DEAD TRAINING START DATE: _____ April 10 _____ (Subtract Line 2 & 3 from Line 1)

Phase 2: The Stack & Compute Target

1. Base architecture backbone, include encoders etc., add as many lines as you need:
_____ VQ-VAE for chance encoder, ResNets for prediction and dynamics nets _____
Param Count: 6 Million
2. Target Compute Hardware:
☒ [X] Local / Lab Node
☐ [] Cloud Rental (e.g., Lambda/AWS)
3. Specific GPU(s): _____ NVIDIA RTX 5090 _____
Total VRAM Available: 32 GB

Phase 3: The Memory Math (The VRAM Ledger)

Reference: 1B Parameters $\approx$ 2GB (Weights) + 2GB (Gradients) + 12GB (AdamW Optimizer States)

1. Model Weights (BF16): 0.012 GB
2. Gradients (BF16): 0.012 GB
3. Optimizer States: 0.072 GB
4. KV Cache & Activations Reserve: 2 GB
5. TOTAL REQUIRED VRAM: 2.1 GB

The Strategy: Is your Total Required VRAM > Total Available VRAM?

☐ [] Yes ☒ [X] No

If Yes, what is your exact mitigation strategy? ☐ FSDP / ZeRO-3 (Sharding across multiple GPUs)

☐ LoRA / QLoRA (Freezing base weights, avoiding optimizer state explosion)

☐ I am shrinking my model selection.

☐ Something else (explain):

Phase 4: The Data Ingress (The I/O Bottleneck)

1. Dataset Size: _____ 2.0 _____ GB / _____ 200,000 _____ Total Samples

2. Format:

☒ Raw JPEGs/PNGs/trajectories/etc. in a folder (High I/O Risk)

☐ WebDataset / Sharded .tar

☐ Something else

3. Storage Location:

☒ Local NVMe SSD

☐ Network Drive

☐ S3/GCS Bucket

☐ Something else

If using raw images or other large data quantities over a network or standard SSD, how will you prevent your dataloader from starving your GPUs?

The biggest bottleneck if self-play compute for VOI MuZero. I will handle this by balancing the number of simulations in a given tree search and the batch size sampling from the game buffer.

Phase 5: Red Team Audit (Partner Review)

Reviewing Engineer: _____ N/A _____

Look at your partner's ledger. Find the fatal flaw. Are they underestimating training time? Do they have a 40GB model fitting into 24GB of VRAM? Are they trying to load 2 million JPEGs sequentially?

The Fatal Flaw / Biggest Risk:

Phase 6: The 72-Hour Contract

What exact, granular engineering task will you complete in the next 72 hours to unblock these risks? (e.g., "Write dummy dataloader script to verify throughput," "Run a 10-step overfitting batch to check VRAM usage.")

Deliverable by Monday: Stress test the GPU on default Stochastic MuZero training on the cartpole problem.